

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Non-Traditional Computing

Instructor: Prof. Joshua San Miguel

In-Cache Data-Level Parallelism

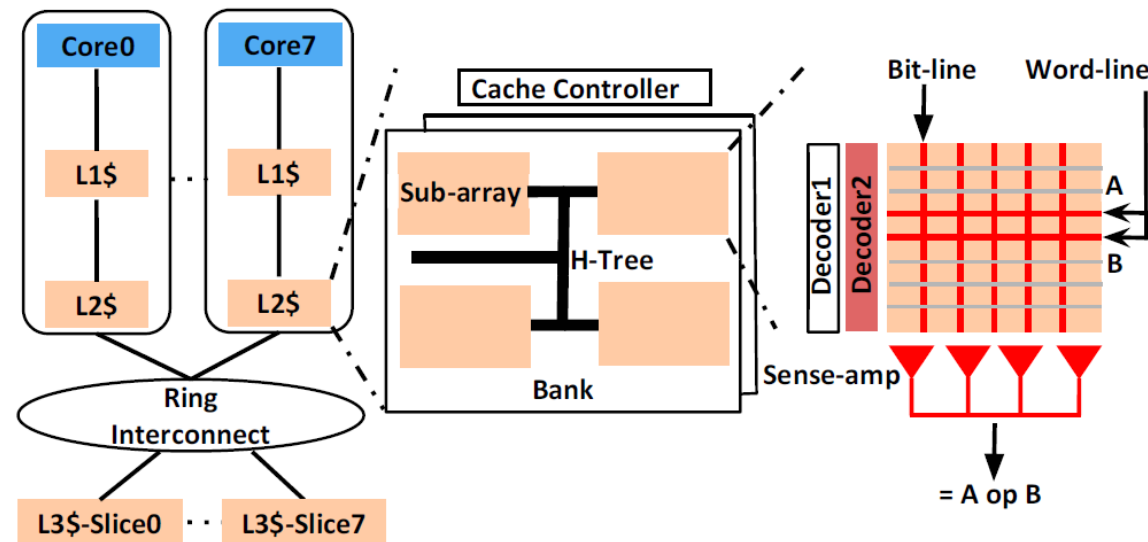
S. Aga, S. Jeloka, A. Subramaniyan, S. Narayanasamy, D. Blaauw and R. Das, *Compute Caches*, HPCA, 2017

How can we transform caches into large vector processing units?

Compute Caches

General-purpose bit-line computing in SRAM caches:

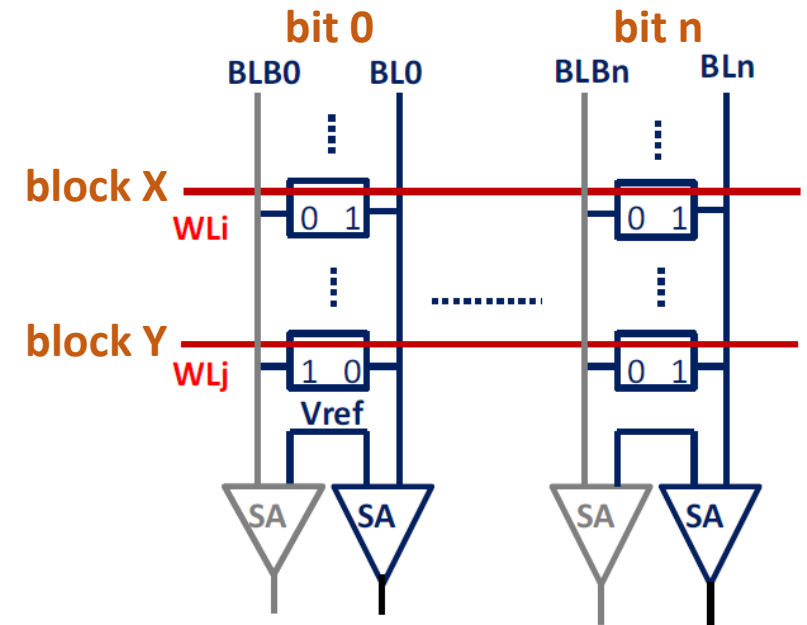
- **Bit-line computing:** simultaneously activate multiple word-lines to perform bulk *AND* and *NOR* operations on the bit-lines
- Extend this to support more bulk operations (e.g., copy, search, compare)
- Provide simple mechanisms for maintaining **operand locality** (i.e., operands must be in same cache partition/sub-array such that they share bit-lines)



Compute Caches – Bit-Line Computing

Activate multiple word-lines (i.e., blocks),
pre-charge all bit-lines (i.e., bits in block)
to 1, then sense the bit-lines:

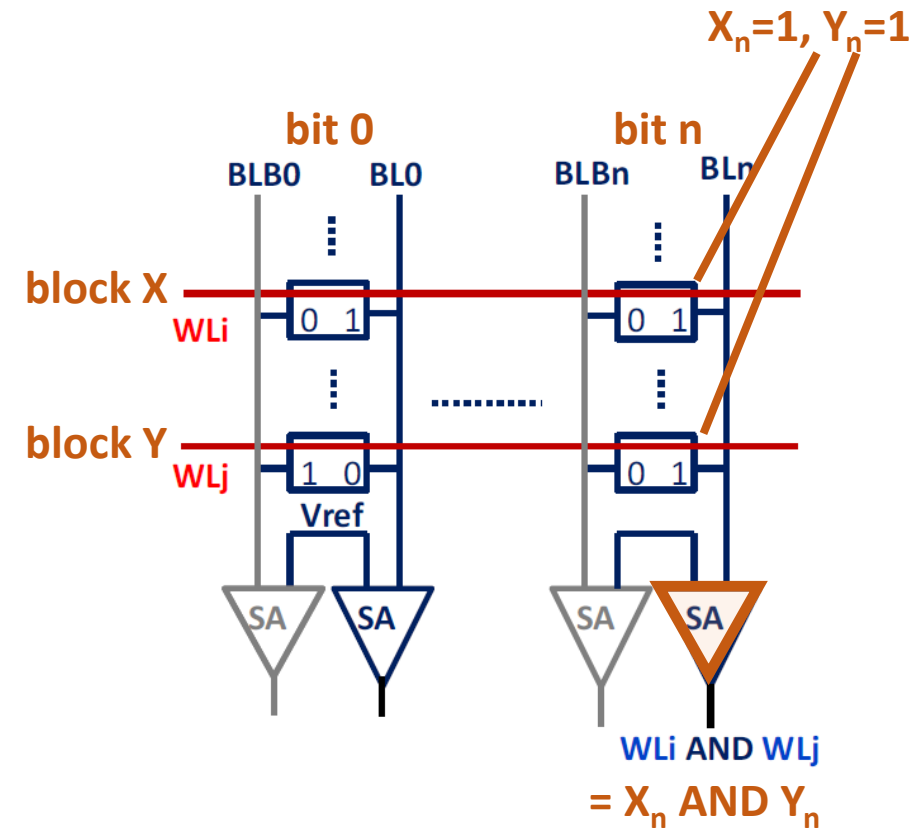
- Sensing a bit-line returns 1 only if the bits of the activated word-lines are all 1; it returns 0 if any one of them are 0.
- **Sensing bit-line (BL):** performs AND
- **Sensing bit-line bar (BLB):** performs NOR



Compute Caches – Bit-Line Computing

Activate multiple word-lines (i.e., blocks),
pre-charge all bit-lines (i.e., bits in block)
to 1, then sense the bit-lines:

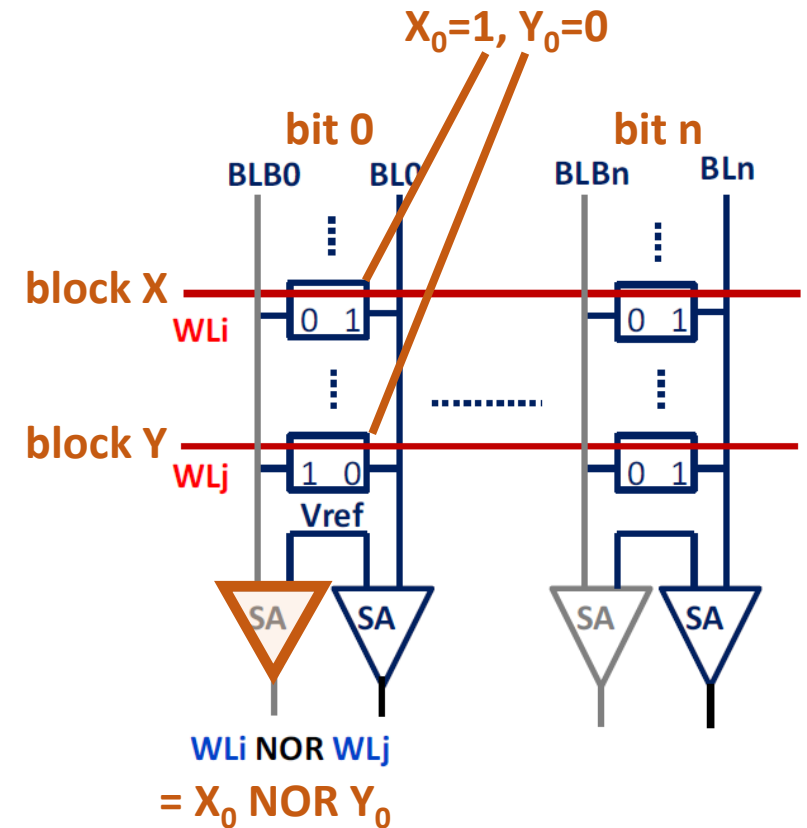
- Sensing a bit-line returns 1 only if the bits of the activated word-lines are all 1; it returns 0 if any one of them are 0.
- **Sensing bit-line (BL):** performs AND
- **Sensing bit-line bar (BLB):** performs NOR



Compute Caches – Bit-Line Computing

Activate multiple word-lines (i.e., blocks),
pre-charge all bit-lines (i.e., bits in block)
to 1, then sense the bit-lines:

- Sensing a bit-line returns 1 only if the bits of the activated word-lines are all 1; it returns 0 if any one of them are 0.
- **Sensing bit-line (BL):** performs AND
- **Sensing bit-line bar (BLB):** performs NOR



Compute Caches – Bit-Line Computing

Opcode	Src1	Src2	Dest	Size	Description
cc_copy	a	-	b	n	$b[i] = a[i]$
cc_buz	a	-	-	n	$a[i] = 0$
cc_cmp	a	b	r	n	$r[i] = (a[i] == b[i])$
cc_search	a	k	r	n	$r[i] = (a[i] == k)$
cc_and	a	b	c	n	$c[i] = a[i] \& b[i]$
cc_or	a	b	c	n	$c[i] = (a[i] b[i])$
cc_xor	a	b	c	n	$c[i] = a[i] \oplus b[i]$
cc_clmulX	a	b	c	n	$c_i = \oplus(a[i] \& b[i])$
cc_not	a	-	b	n	$b[i] = !(a[i])$
a,b,c,k: addresses			r:register	$\forall i, i \in [1, n], X = [64/128/256]$	

Add additional logic at the sense-amplifiers to support more ops

- Feed sense-amp output back into bit-lines to write results back into cache

Compute Caches – Bit-Line Computing

Opcode	Src1	Src2	Dest	Size	Description
cc_copy	a	-	b	n	$b[i] = a[i]$
cc_buz	a	-	-	n	$a[i] = 0$
cc_cmp	a	b	r	n	$r[i] = (a[i] == b[i])$
cc_search	a	k	r	n	$r[i] = (a[i] == k)$
cc_and	a	b	c	n	$c[i] = a[i] \& b[i]$
cc_or	a	b	c	n	$c[i] = (a[i] b[i])$
cc_xor	a	b	c	n	$c[i] = a[i] \oplus b[i]$
cc_clmulX	a	b	c	n	$c_i = \oplus(a[i] \& b[i])$
cc_not	a	-	b	n	$b[i] = !(a[i])$
a,b,c,k: addresses			r:register	$\forall i, i \in [1, n], X = [64/128/256]$	

Add additional logic at the sense-amplifiers to support more ops

- Feed sense-amp output back into bit-lines to write results back into cache
- ***How to perform XOR?***

Compute Caches – Bit-Line Computing

Opcode	Src1	Src2	Dest	Size	Description
cc_copy	a	-	b	n	$b[i] = a[i]$
cc_buz	a	-	-	n	$a[i] = 0$
cc_cmp	a	b	r	n	$r[i] = (a[i] == b[i])$
cc_search	a	k	r	n	$r[i] = (a[i] == k)$
cc_and	a	b	c	n	$c[i] = a[i] \& b[i]$
cc_or	a	b	c	n	$c[i] = (a[i] b[i])$
cc_xor	a	b	c	n	$c[i] = a[i] \oplus b[i]$
cc_clmulX	a	b	c	n	$c_i = \oplus(a[i] \& b[i])$
cc_not	a	-	b	n	$b[i] = !(a[i])$
a,b,c,k: addresses			r:register	$\forall i, i \in [1, n], X = [64/128/256]$	

Add additional logic at the sense-amplifiers to support more ops

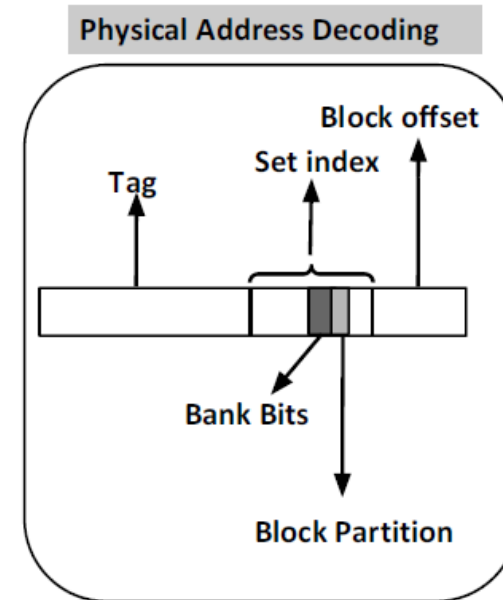
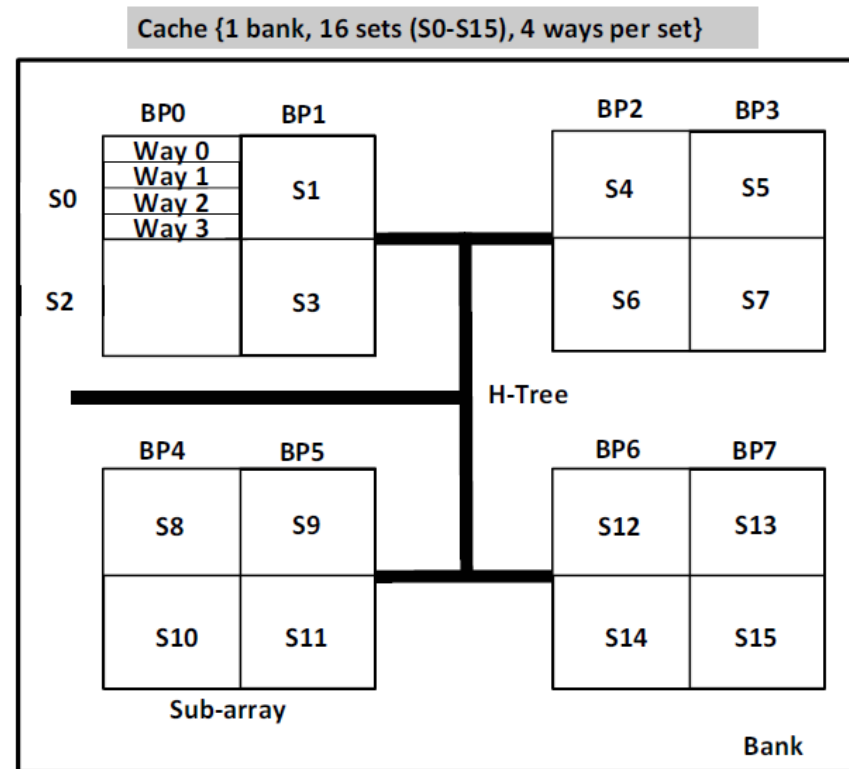
- Feed sense-amp output back into bit-lines to write results back into cache
- ***How to perform XOR? = BL NOR BLB = (x AND y) NOR (x NOR y)***



Compute Caches – Operand Locality

Block partition (BP): group of blocks that share bit-lines in cache

- Cache consists of banks; each bank consists of BPs
- As long as two blocks have same bank+BP bits (subset of set bits), they share bit-lines
- $x \text{ OP } y$ requires that x and y have the same bank+BP+block offset bits

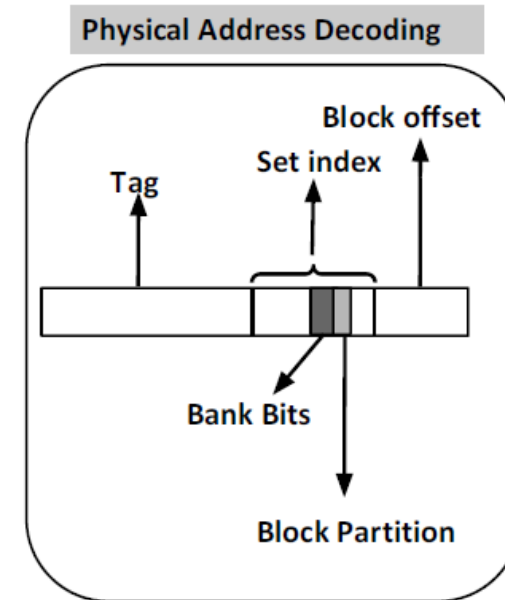
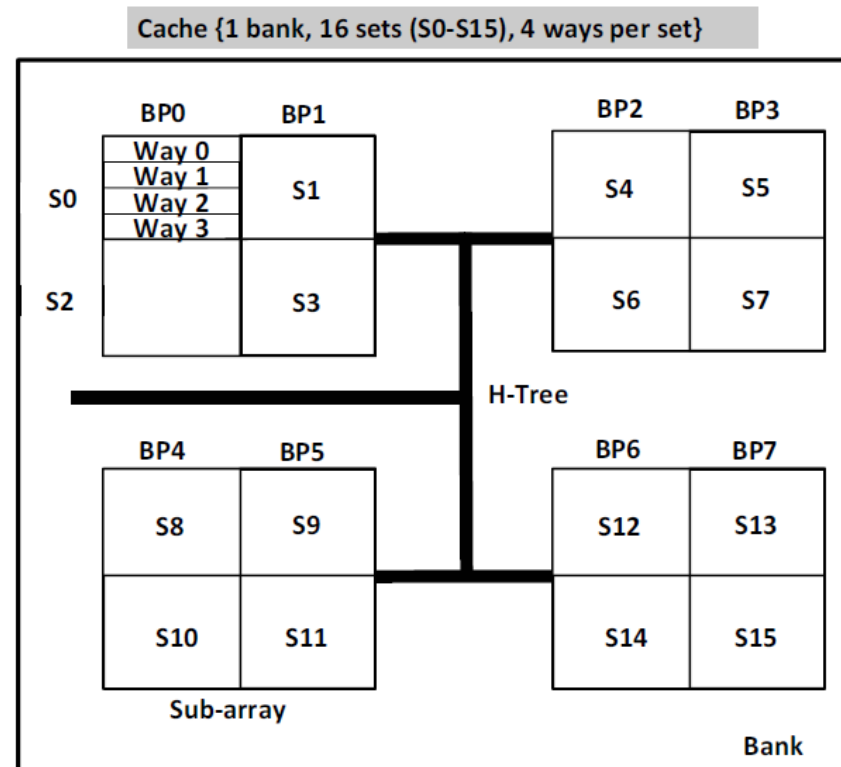


Compute Caches – Operand Locality

Block partition (BP): group of blocks that share bit-lines in cache

- Cache consists of banks; each bank consists of BPs
- As long as two blocks have same bank+BP bits (subset of set bits), they share bit-lines
- $x \text{ OP } y$ requires that x and y have the same bank+BP+block offset bits

e.g., Assuming 64B blocks and set index = $\text{addr}[9:6]$, which addr bits for BP?

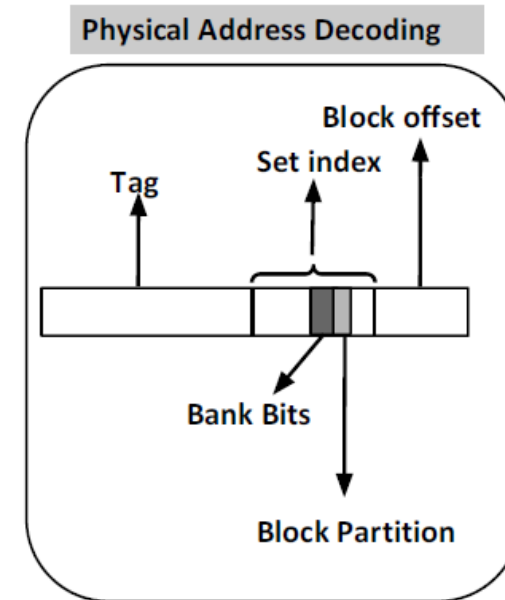
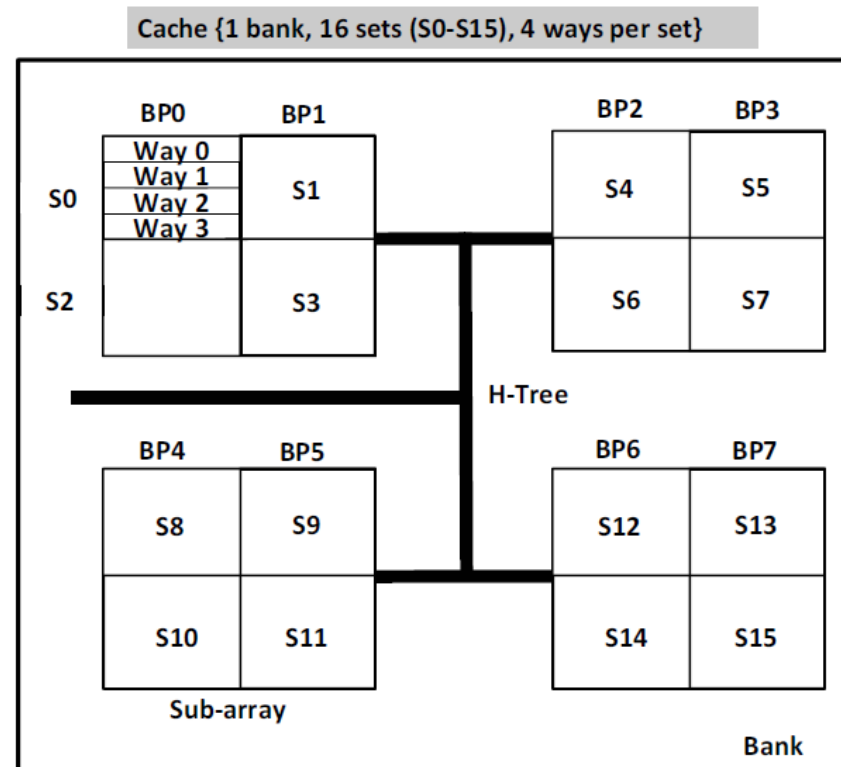


Compute Caches – Operand Locality

Block partition (BP): group of blocks that share bit-lines in cache

- Cache consists of banks; each bank consists of BPs
- As long as two blocks have same bank+BP bits (subset of set bits), they share bit-lines
- $x \text{ OP } y$ requires that x and y have the same bank+BP+block offset bits

e.g., Assuming 64B blocks and set index = $\text{addr}[9:6]$, which addr bits for BP?
 $\text{addr}[9:8,6]$



Compute Caches – Operand Locality

Block partition (BP): group of blocks that share bit-lines in cache

- Cache consists of banks; each bank consists of BPs
- As long as two blocks have same bank+BP bits (subset of set bits), they share bit-lines
- x OP y requires that x and y have the same bank+BP+block offset bits
- **Since bank+BP+block offset bits usually within page offset bits, SW maps blocks such that operands are assigned the same page offset**

Cache	Banks	BP	Block size	Min. address bits match
L1-D	2	2	64	8
L2	8	2	64	10
L3-slice	16	4	64	12

if 4KB pages, SW places x and y at the same offset but in different pages



Computational Skimming

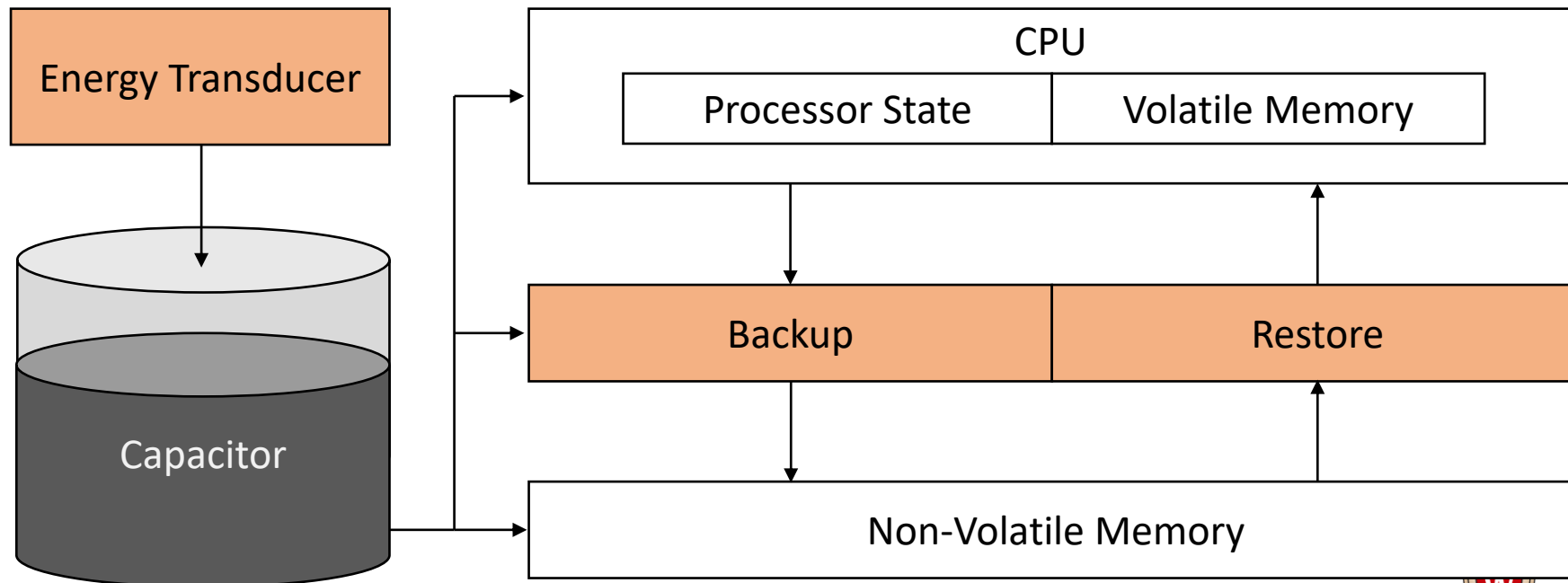
K. Ganesan, J. San Miguel and N. Enright Jerger, *The What's Next Intermittent Computing Architecture*, HPCA, 2019

How can we run programs in an interruptible fashion on batteryless systems?

Batteryless Systems

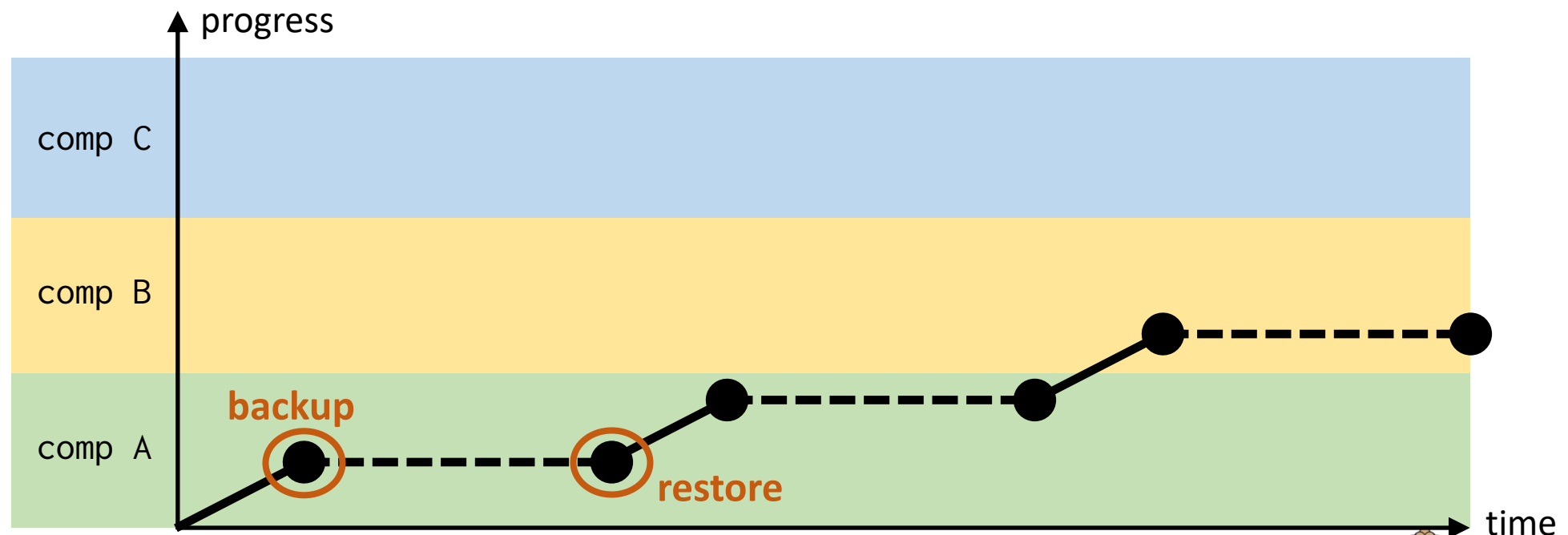
Intermittent Computing: May lose power anytime during program

- Must *back up* the current program state (i.e., PC, register file) and *restore it* after regaining sufficient power
 - **Non-volatile memory:** retains data even upon a power loss (e.g., FRAM, STT-RAM)



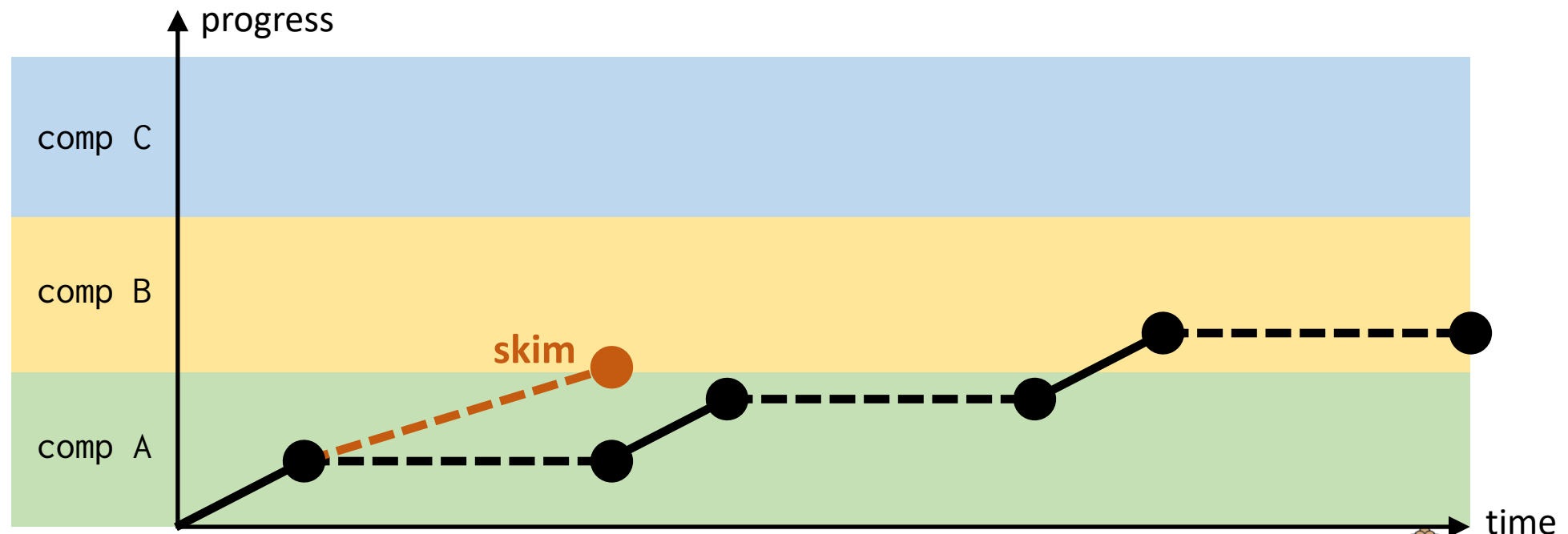
Computational Skimming

Insight: Decouple backup from restore point



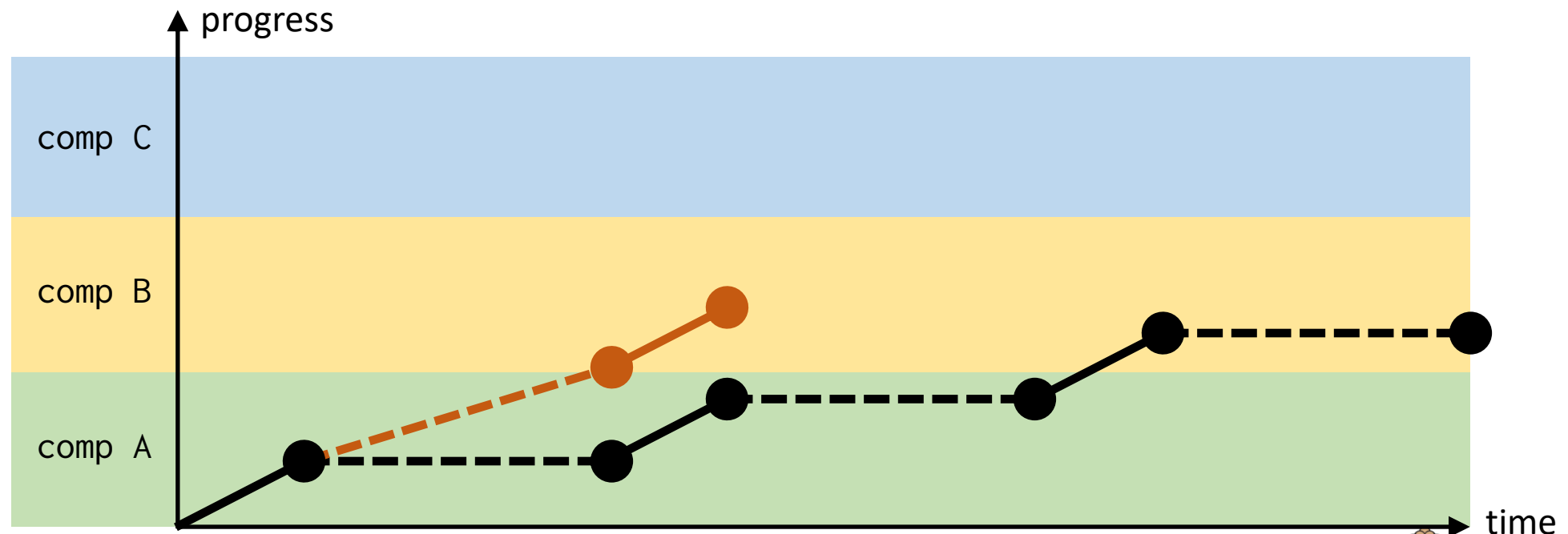
Computational Skimming

Insight: Decouple backup from restore point



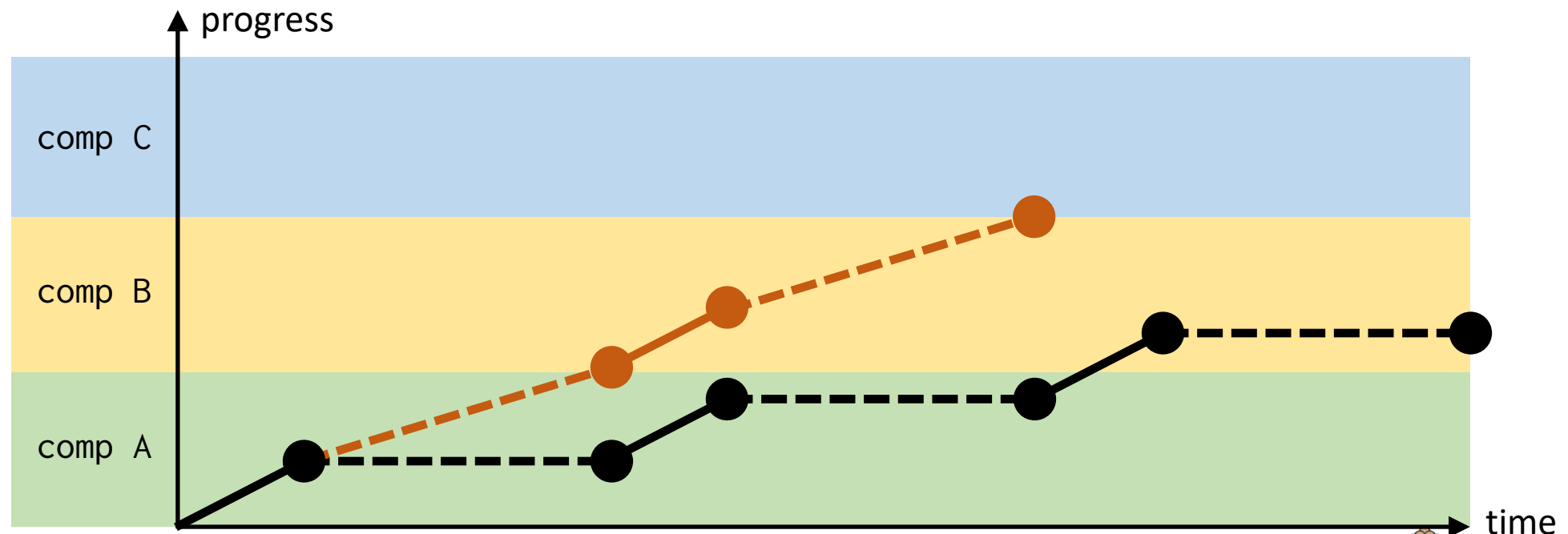
Computational Skimming

Insight: Decouple backup from restore point



Computational Skimming

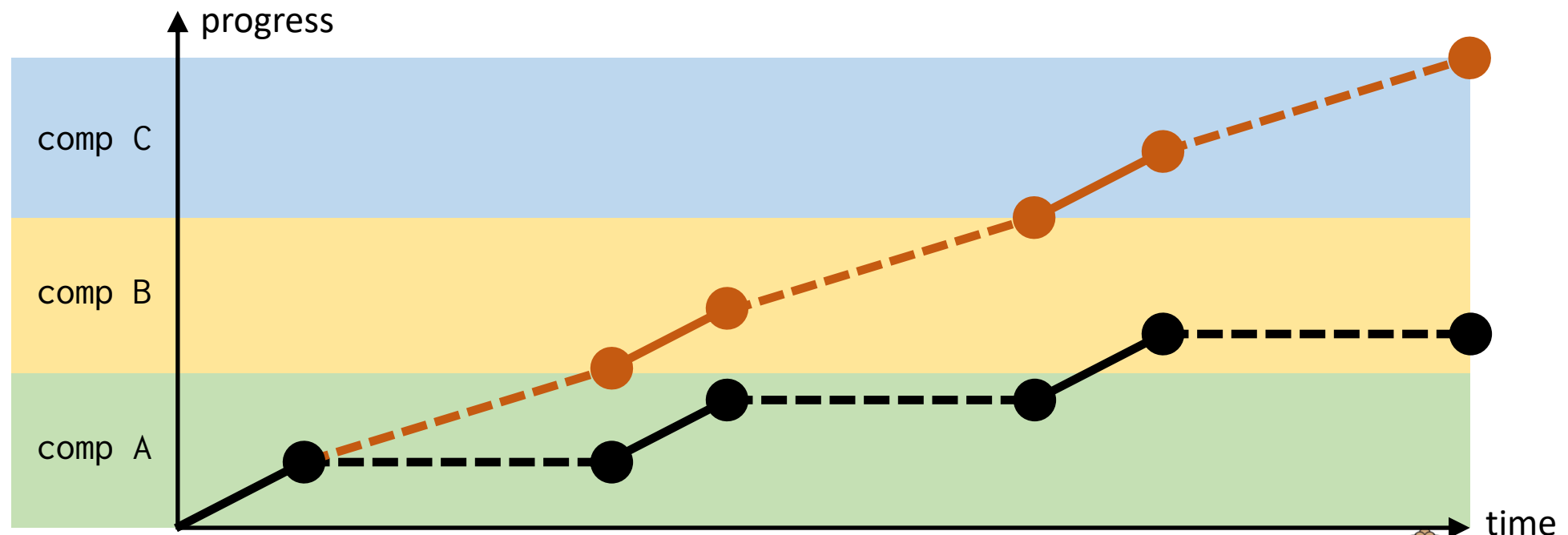
Insight: Decouple backup from restore point



Computational Skimming

Insight: Decouple backup from restore point

Computational skimming: Quality of program output scales with how much energy we can afford



Computational Skimming

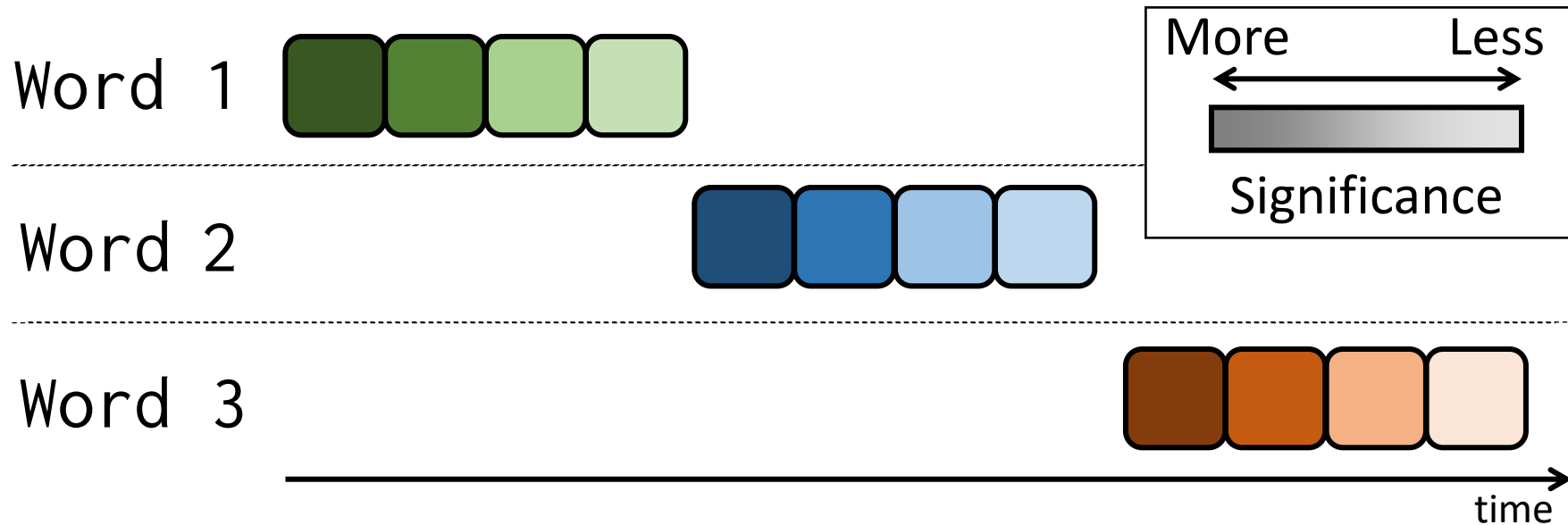
What's Next architecture for computational skimming [HPCA'19]:

- **Skim points:** Backups for intermediate approximate results
 - Jumps to future PC restore point if power is lost
- Leverages parallelism at the subword granularity

Computational Skimming – Subword Pipelining

Long-latency operations (e.g., mul):

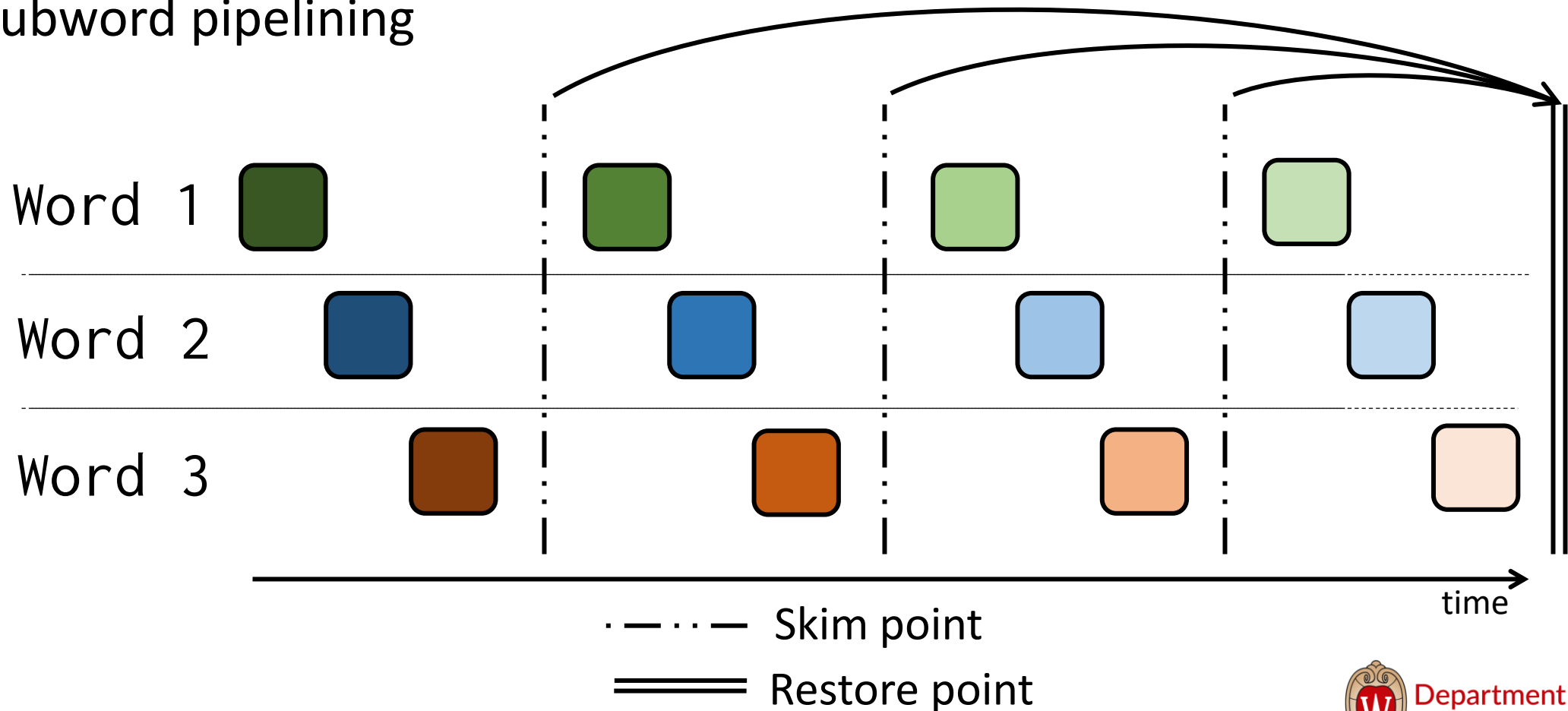
➤ Conventional



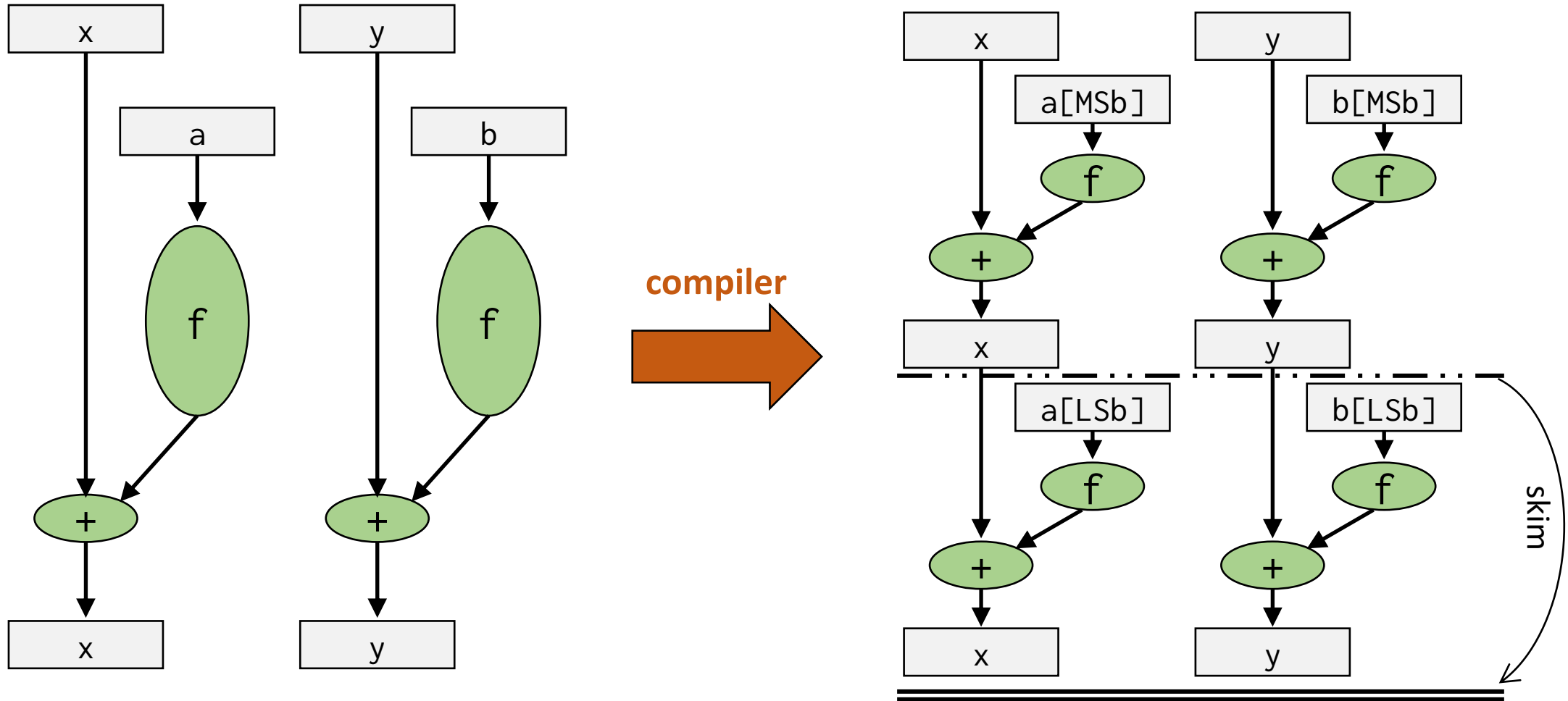
Computational Skimming – Subword Pipelining

Long-latency operations (e.g., mul):

➤ Subword pipelining



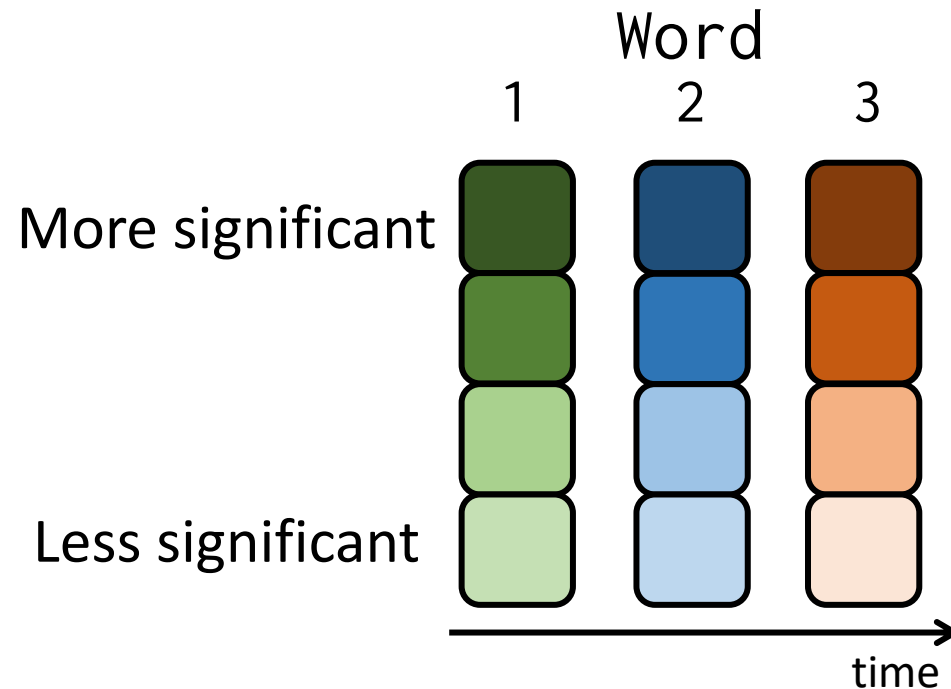
Computational Skimming – Subword Pipelining



Computational Skimming – Subword Vectorization

Short-latency operations (e.g., add):

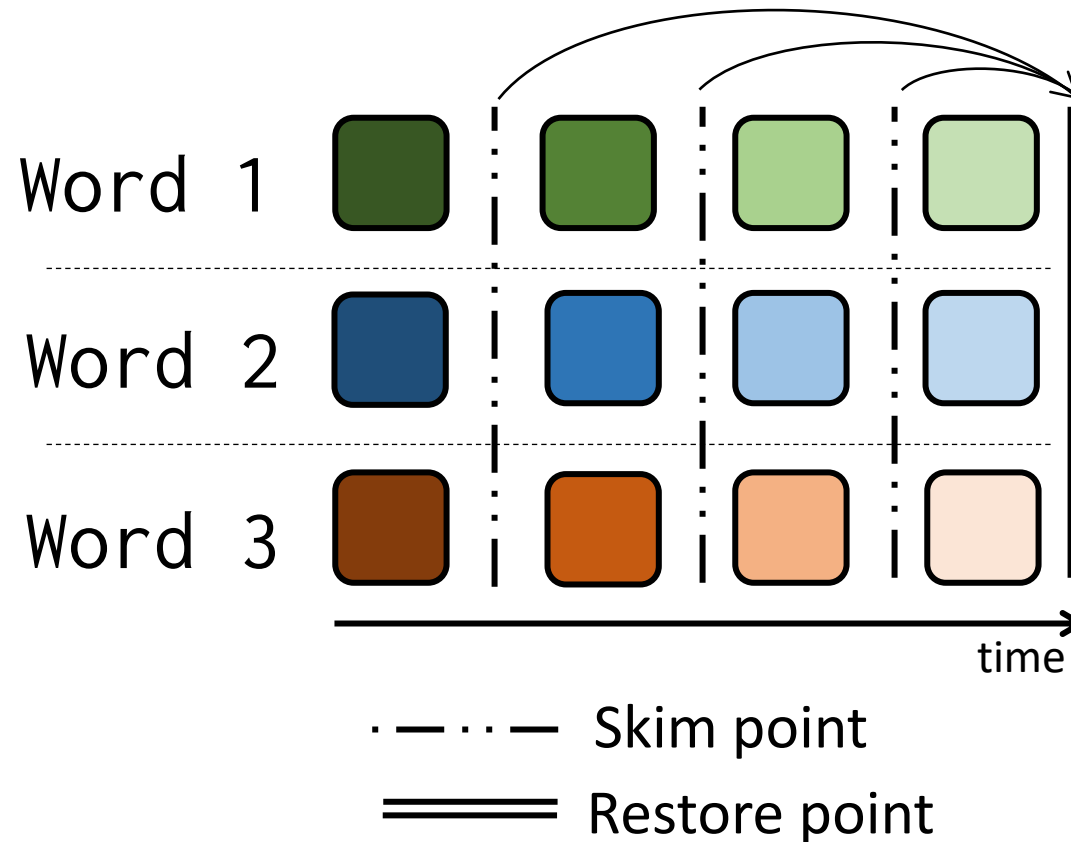
➤ Conventional



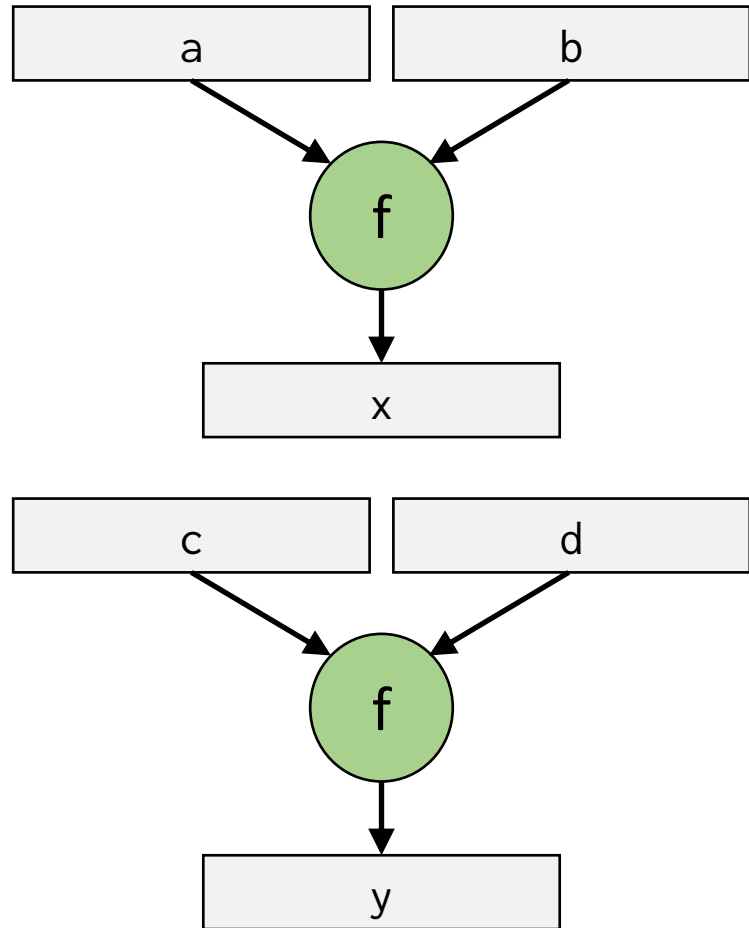
Computational Skimming – Subword Vectorization

Short-latency operations (e.g., add):

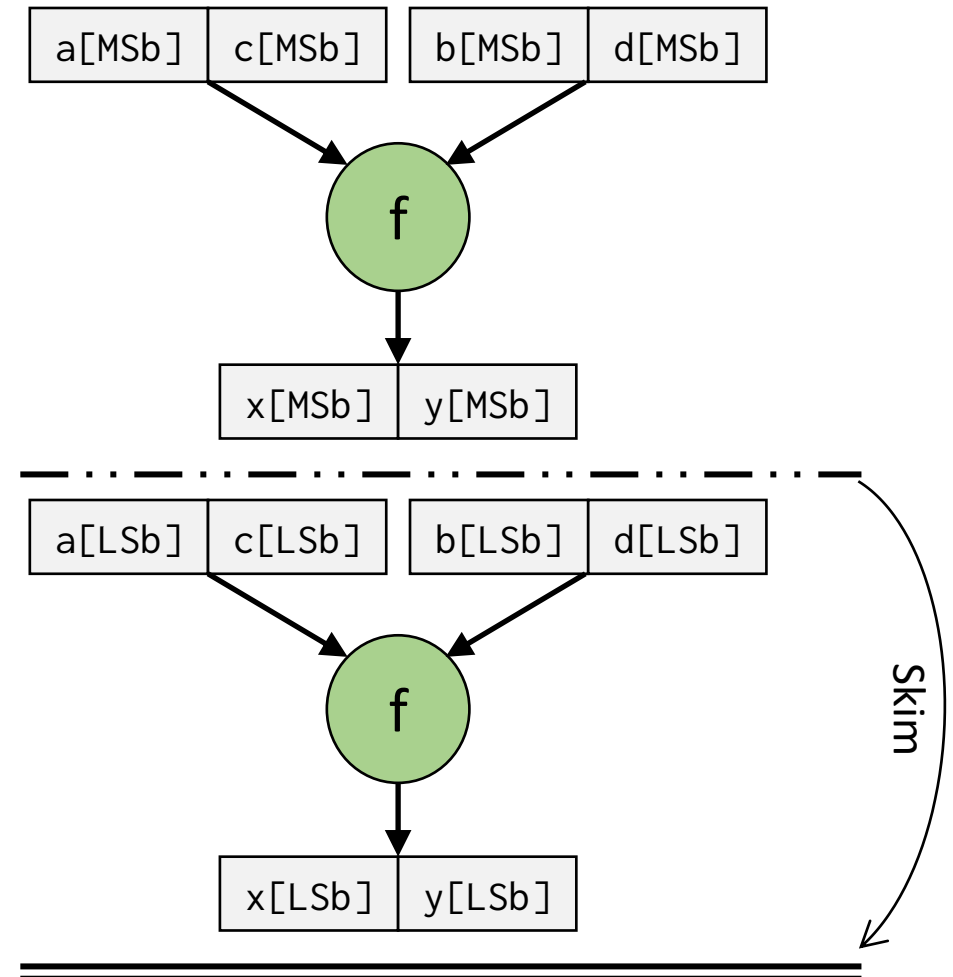
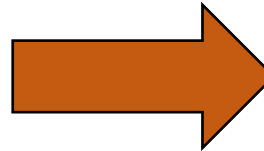
- Subword vectorization



Computational Skimming – Subword Vectorization



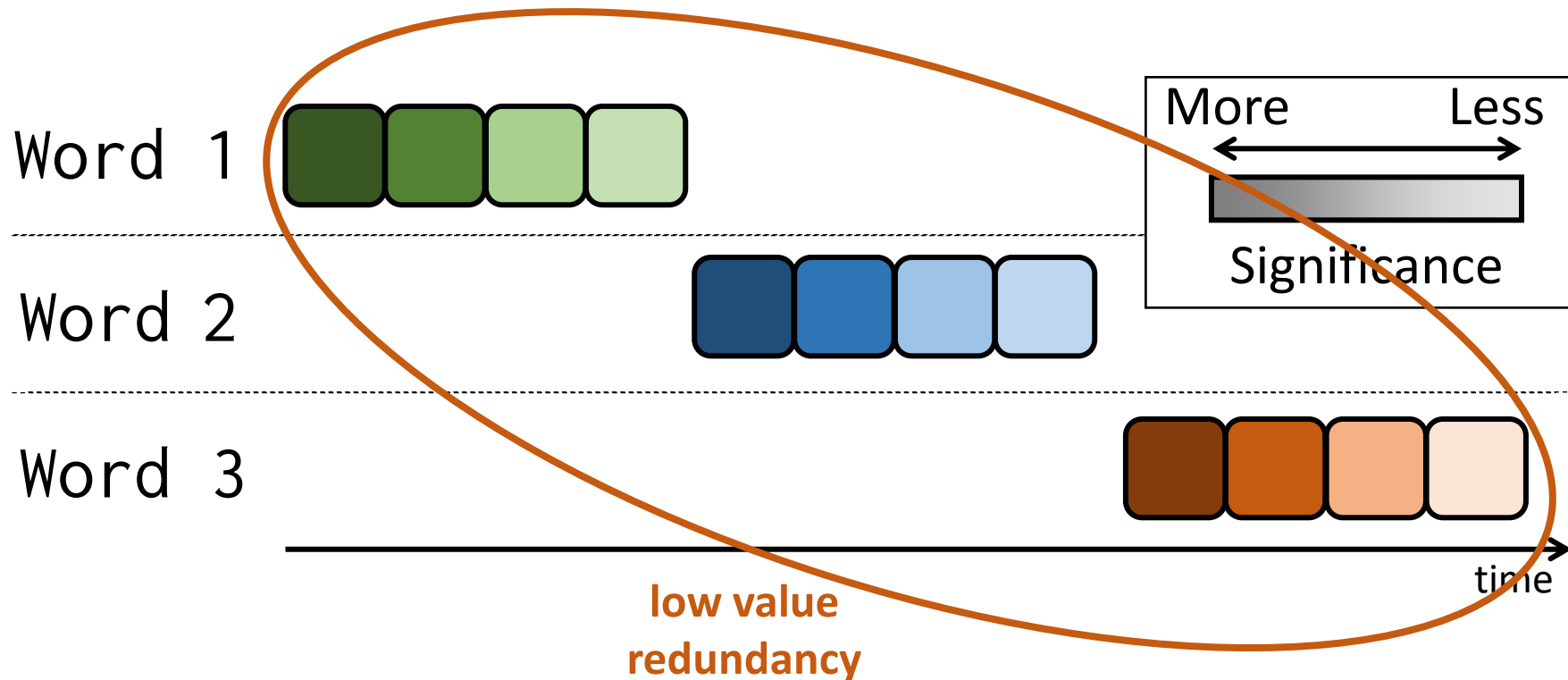
compiler



Computational Skimming – Subword Memoization

In conjunction with subword pipelining:

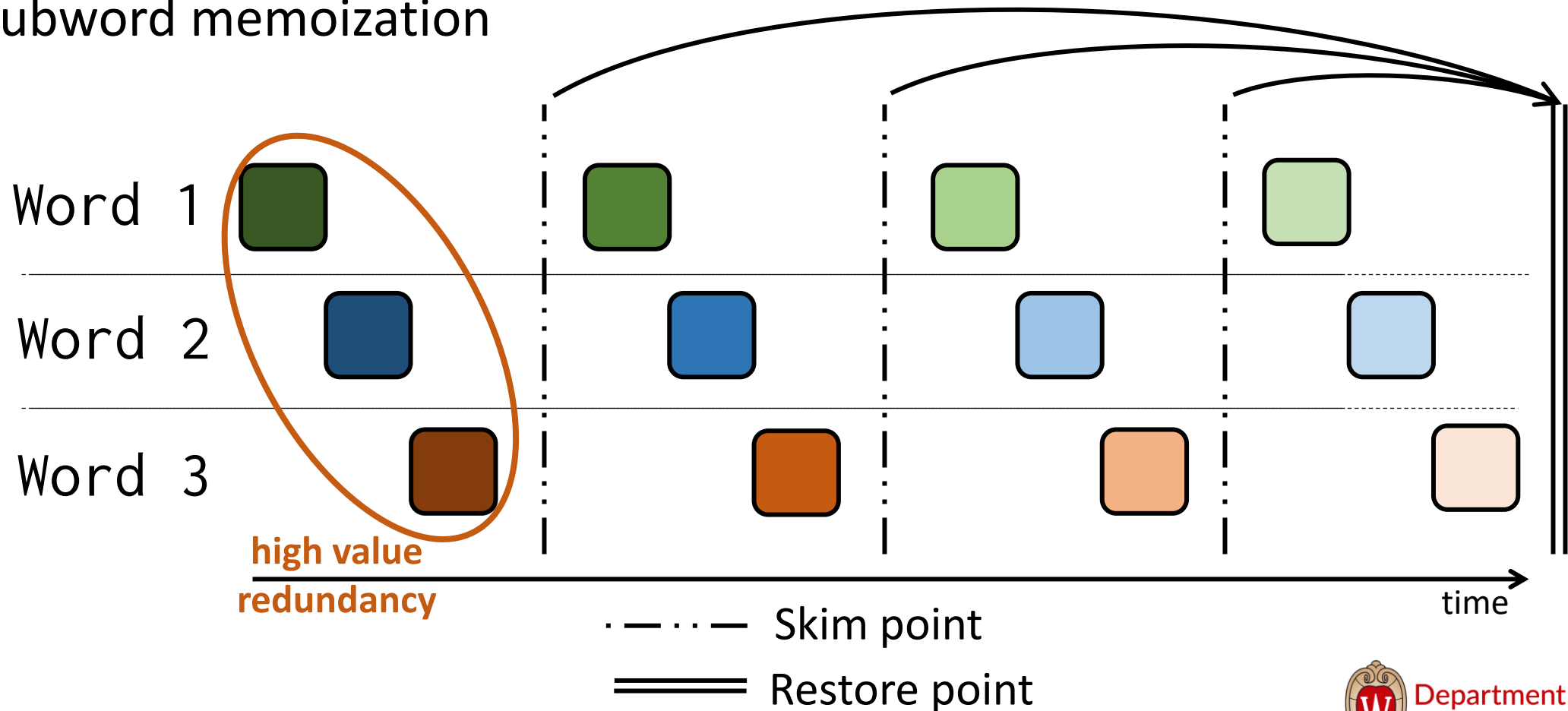
➤ Conventional



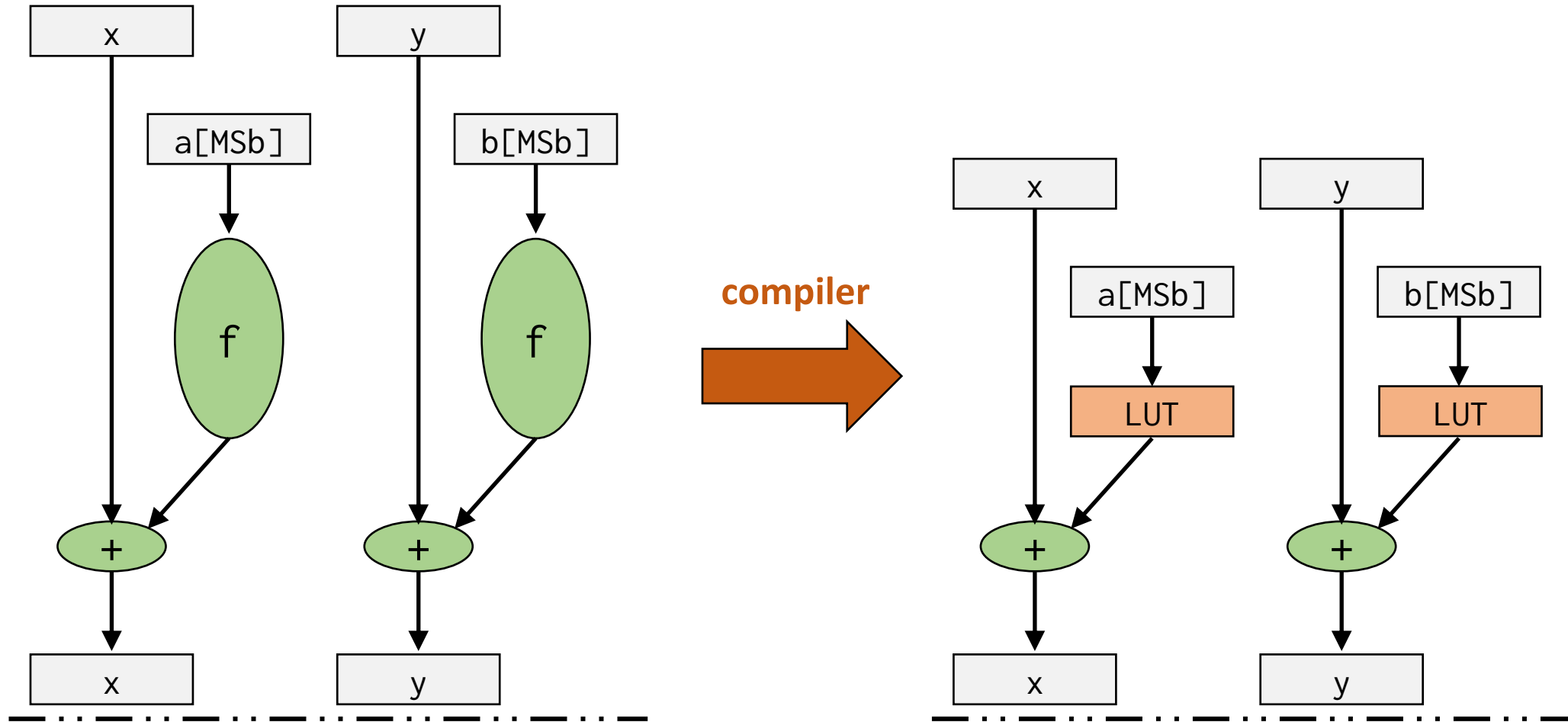
Computational Skimming – Subword Memoization

In conjunction with subword pipelining:

➤ Subword memoization



Computational Skimming – Subword Memoization



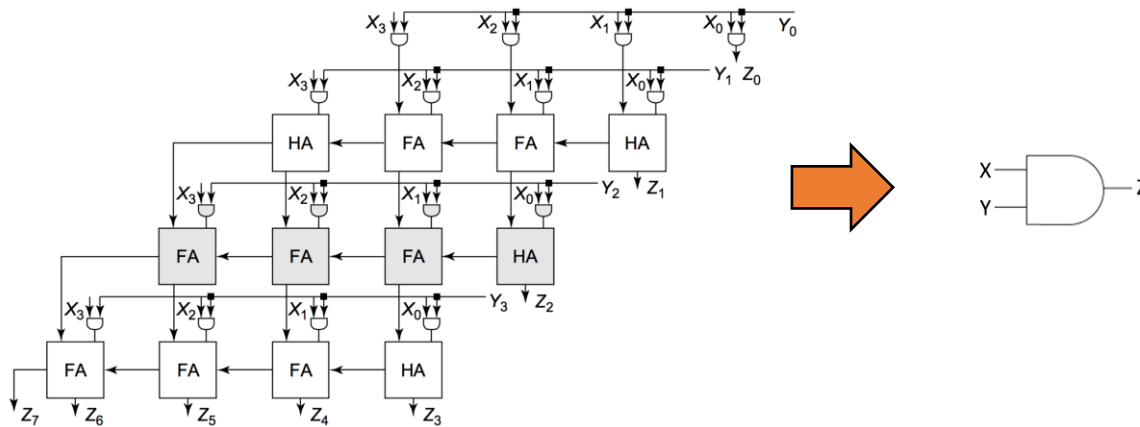
Unary Computing

D. Wu, J. Li, R. Yin, H. Hsiao, Y. Kim and J. San Miguel, *uGEMM: Unary Computing Architecture for GEMM Applications*, ISCA, 2020

How can we perform computations on unary (not binary) data?

Unary Computing

Unary computing encodes data as bitstreams for super cheap HW...



...But it trades off generality and accuracy

Unary Computing

Binary data representation:

- $0.b_{-1}b_{-2}b_{-3}b_{-4} \Rightarrow b_{-1} * 2^{-1} + b_{-2} * 2^{-2} + b_{-3} * 2^{-3} + b_{-4} * 2^{-4}$
- e.g., $0.0110_2 \Rightarrow 0.375_{10}$

Unary data representation:

- $b_{15}b_{14}b_{13}...b_2b_1b_0 \Rightarrow (b_{15} * 1^{15} + b_{14} * 1^{14} + ... + b_1 * 1^1 + b_0 * 1^0) / 16$
- e.g., $1010010010100100_1 \Rightarrow 0.375_{10}$

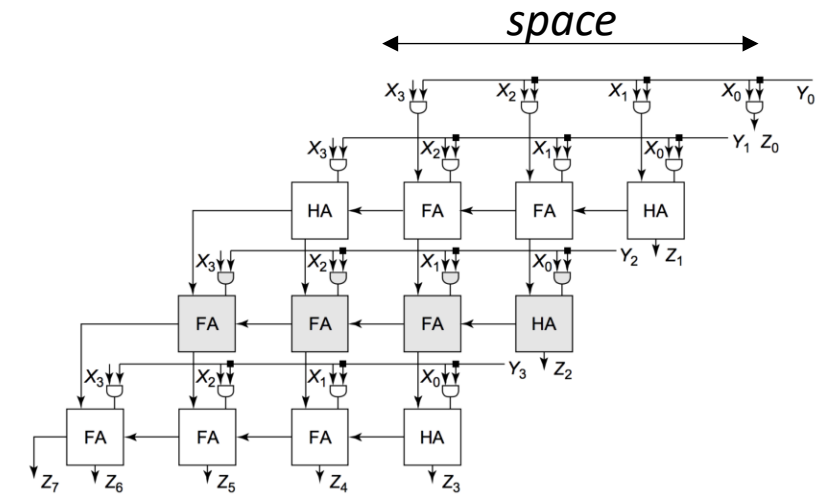
Unary Computing

Binary computing:

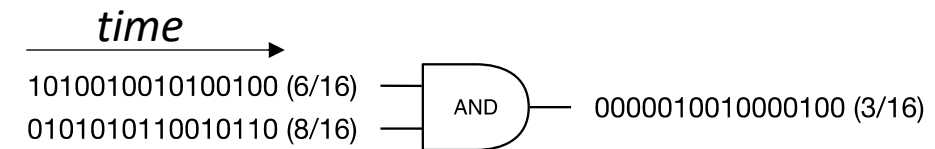
- *Varying* significance in data bits
- *Spatial* domain with *complex* logic

Unary computing:

- *Equal* significance in data bits
- *Temporal* domain with *simple* logic



binary multiplier

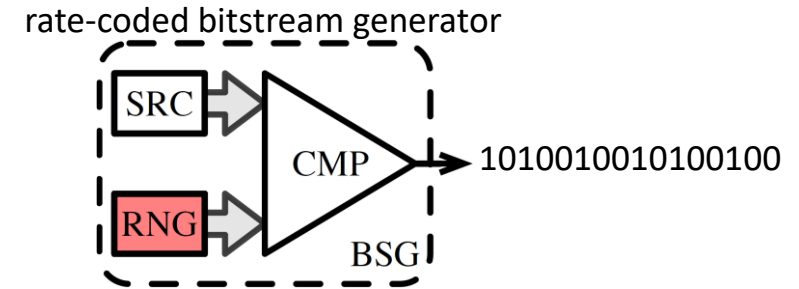


unary multiplier

Unary Computing

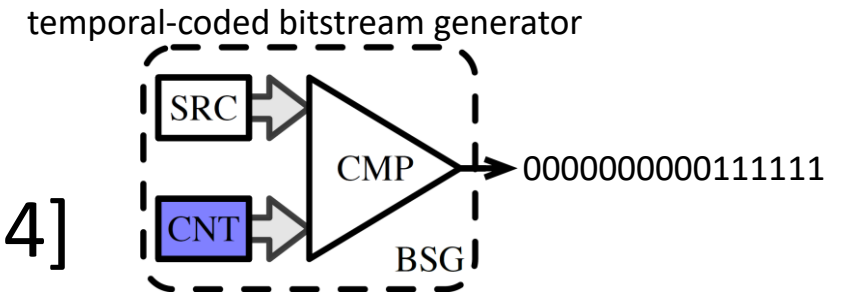
Rate coding:

- Valued by *frequency* of 1s in bitstream
- Used in *stochastic computing*
 - Example apps: LDPC, image processing, DNNs



Temporal coding:

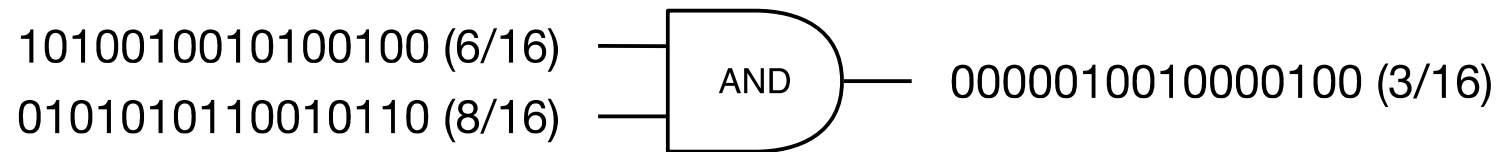
- Valued by *latency* of 1s in bitstream
- Used in *race logic* [Madhavan et al., ISCA'14]
 - Example apps: DNA sequencing, decision trees, sorting



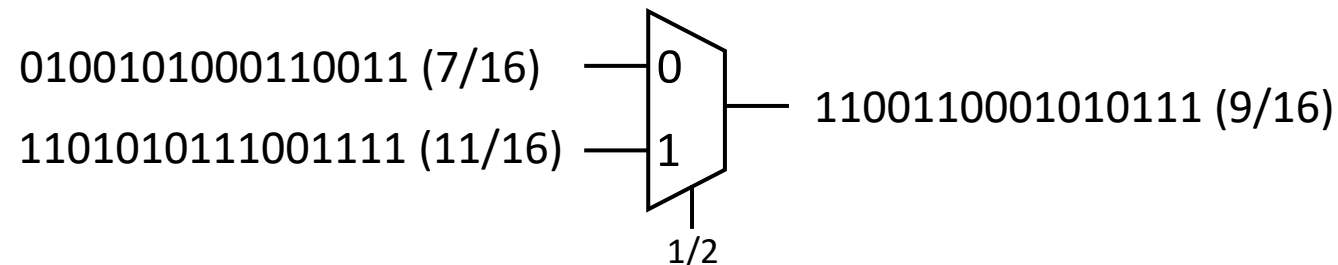
Unary Computing – Rate (Stochastic)

Values are represented as probability of 1s in bitstream

- Use random number generators to produce input bitstreams
- e.g., Multiplication:



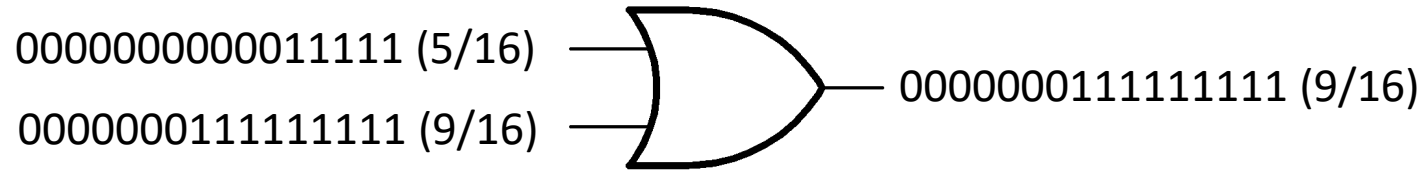
- e.g., Scaled addition:



Unary Computing – Temporal

Values are represented as latency of leading 1s or 0s

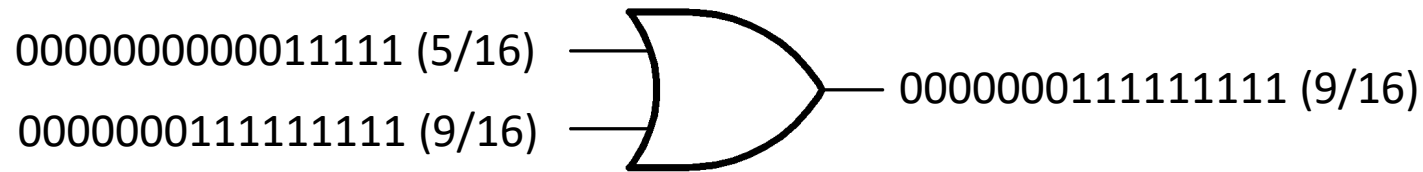
- Use counters or delay registers to produce bitstreams
- Accurate (and deterministic) but supports limited operations
- e.g., Maximum (leading 1s):



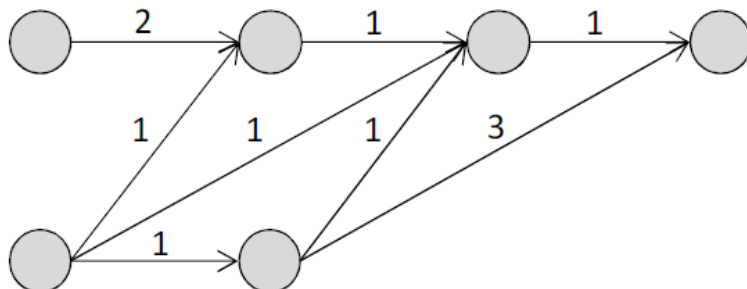
Unary Computing – Temporal

Values are represented as latency of leading 1s or 0s

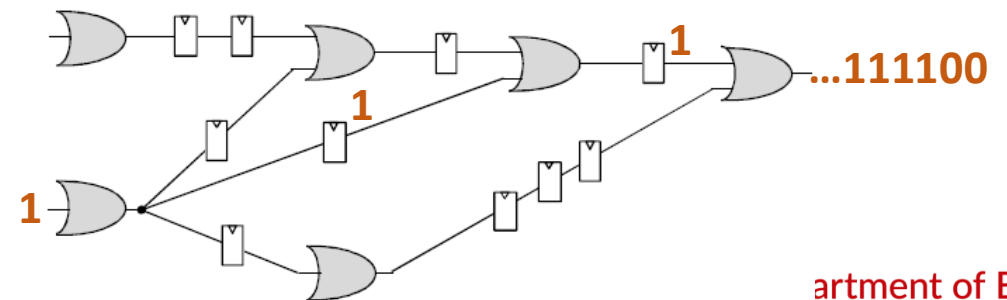
- Use counters or delay registers to produce bitstreams
- Accurate (and deterministic) but supports limited operations
- e.g., Maximum (leading 1s):



- e.g., Shortest path length (leading 0s):



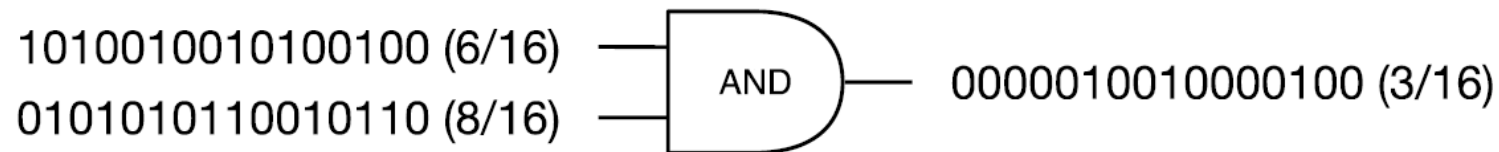
[Madhavan et al., ISCA'14]



Challenge: Correlation

With rate coding, inaccuracies in results due to **correlation**

- Two bitstreams are *positively (negatively) correlated* if 1s and 0s are very aligned (misaligned)
- Need *zero correlation* for bitstream independence
- e.g., Multiplication:



zero correlation: *accurate* result

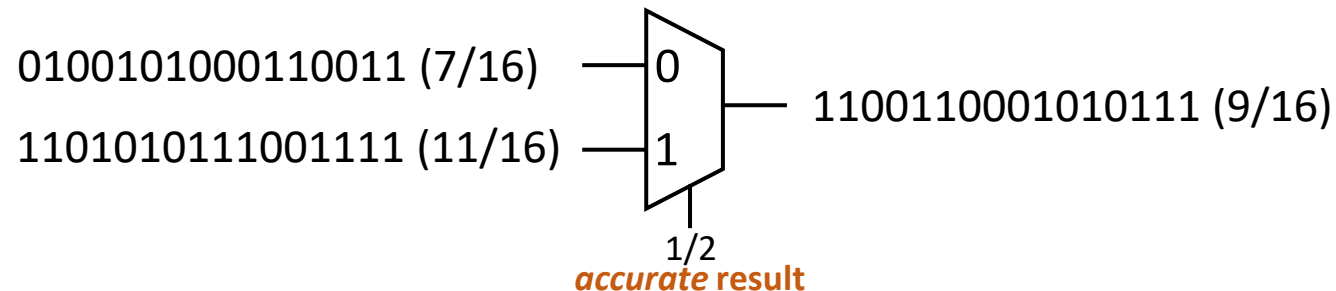


positive correlation: *inaccurate* result

Challenge: Correlation

With rate coding, inaccuracies in results due to **correlation**

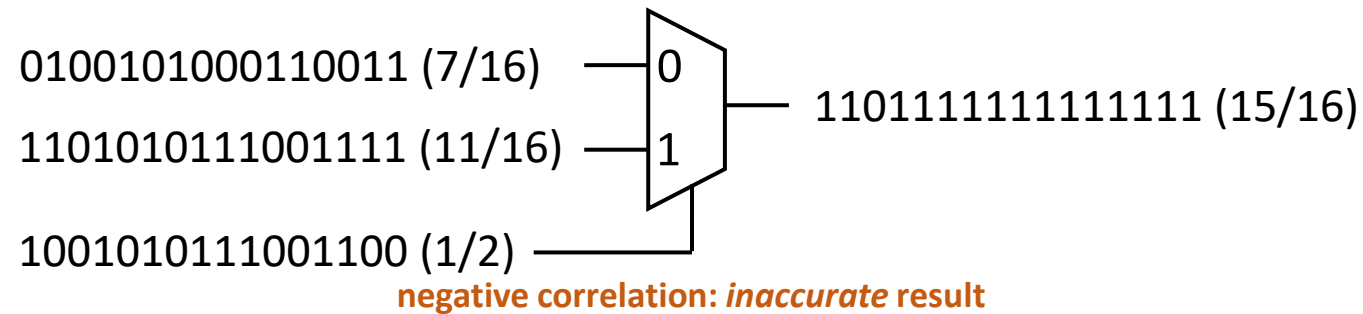
- Two bitstreams are *positively (negatively) correlated* if 1s and 0s are very aligned (misaligned)
- Need *zero correlation* for bitstream independence
- e.g., Scaled addition:



Challenge: Correlation

With rate coding, inaccuracies in results due to **correlation**

- Two bitstreams are *positively (negatively) correlated* if 1s and 0s are very aligned (misaligned)
- Need *zero correlation* for bitstream independence
- e.g., Scaled addition:



Challenge: Correlation?

Revelation #1: Correlation isn't evil; it's just misunderstood

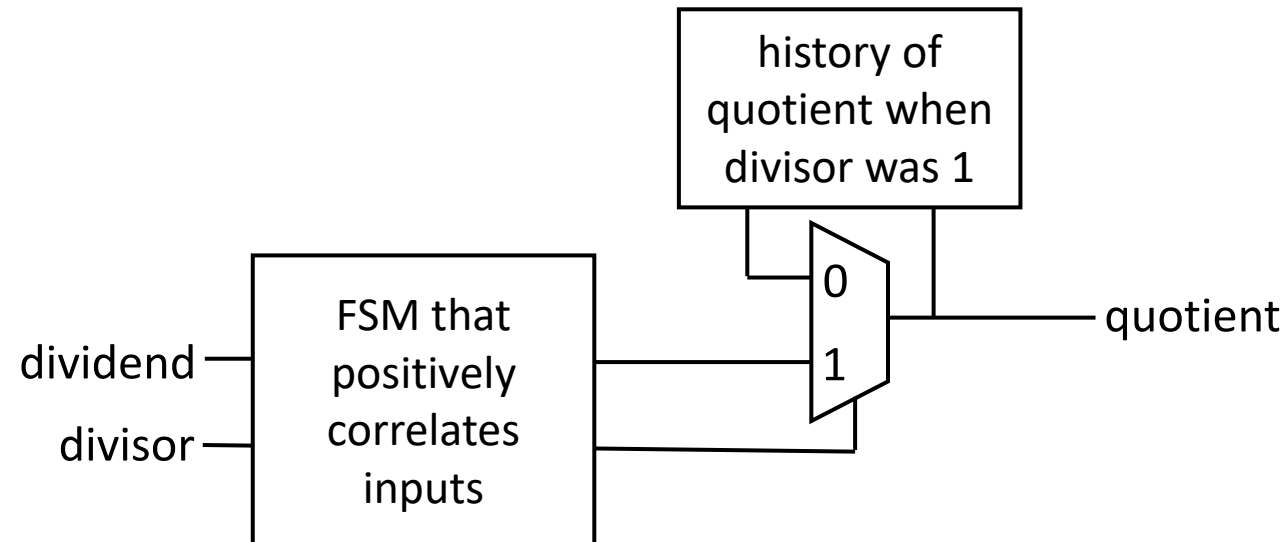
➤ e.g., *Positively correlated* multiplication => minimum:



Challenge: Correlation?

Revelation #1: Correlation isn't evil; it's just misunderstood

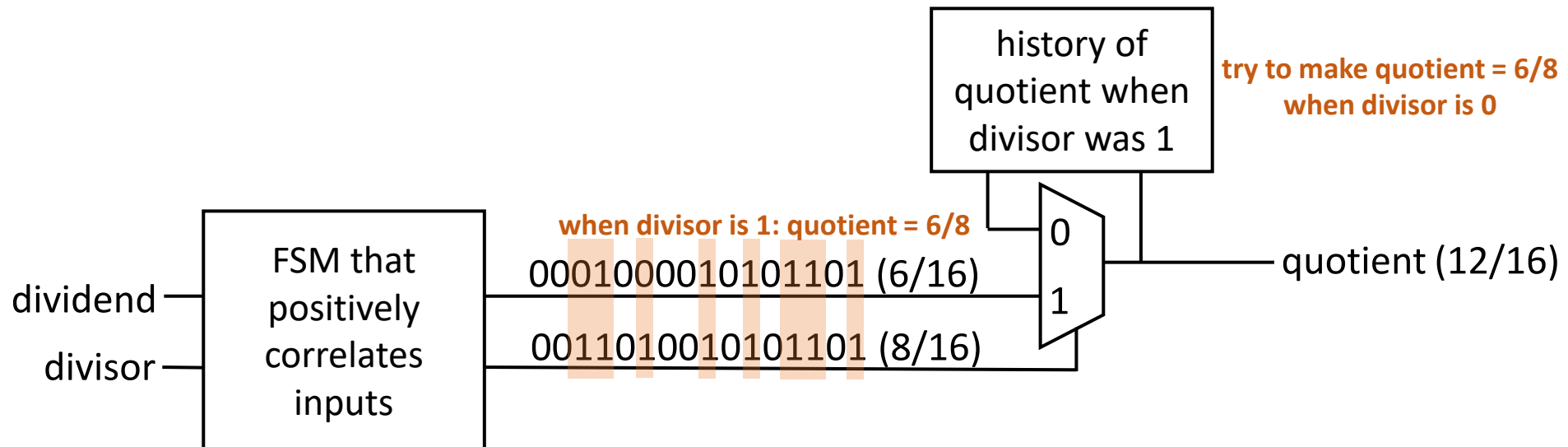
- e.g., Leverage correlation to build unary div/sqrt [DAC 2019]:
 - In stochastic computing, divisor $\geq$ dividend



Challenge: Correlation?

Revelation #1: Correlation isn't evil; it's just misunderstood

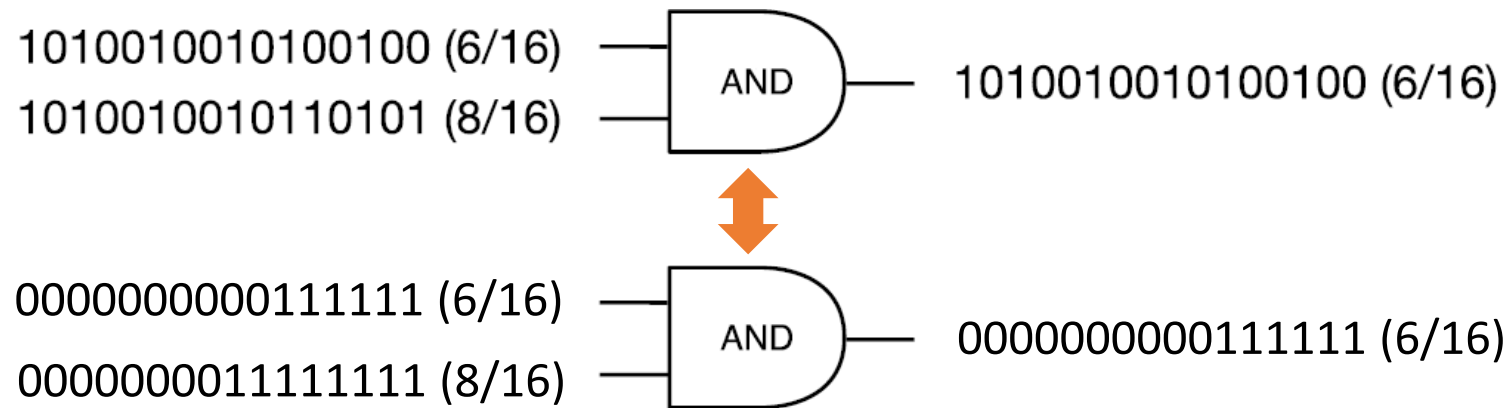
- e.g., Leverage correlation to build unary div/sqrt [DAC 2019]:
 - In stochastic computing, divisor $\geq$ dividend



Challenge: Correlation?

Revelation #2: Stochastic + correlation $\approx$ temporal

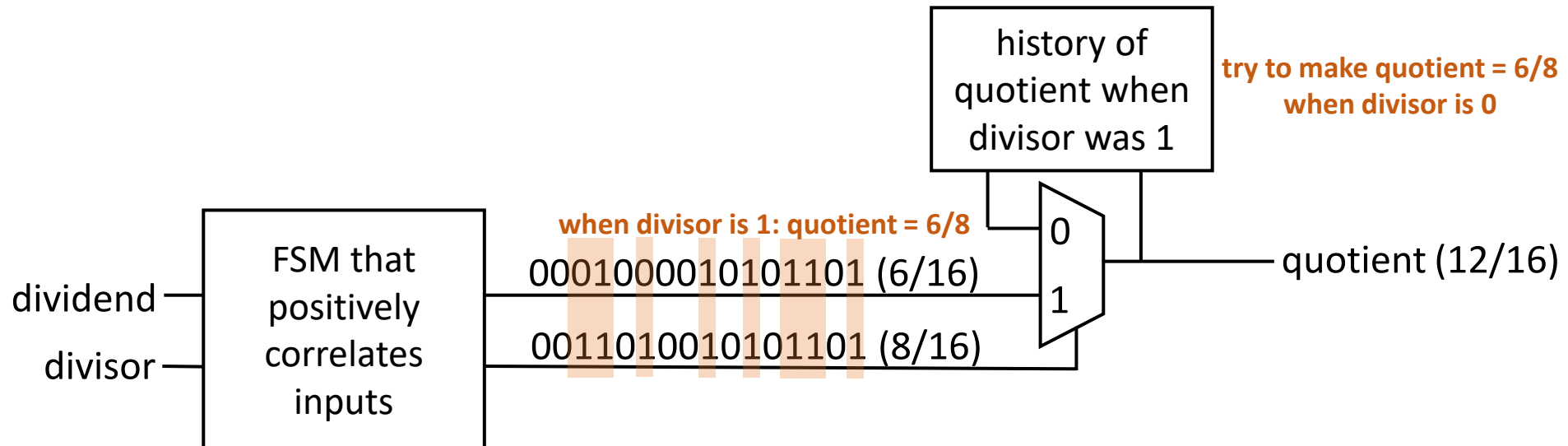
➤ e.g., *Positively correlated* multiplication $\Rightarrow$ minimum:



Challenge: Correlation?

Revelation #2: Stochastic + correlation $\approx$ temporal

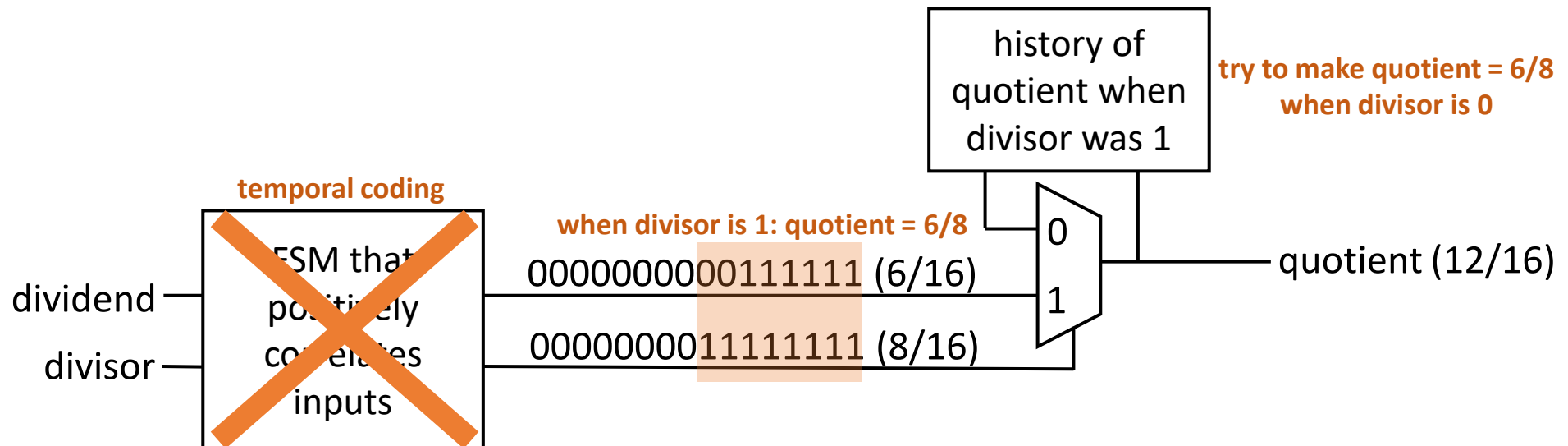
- Temporal coding can yield more accuracy when used on some correlation-based rate-coded circuits



Challenge: Correlation?

Revelation #2: Stochastic + correlation $\approx$ temporal

- Temporal coding can yield more accuracy when used on some correlation-based rate-coded circuits
- **Takeaway:** Unary coding concepts from different worlds can unify



Final Project Deliverables

Final Report

Due: Friday 5/09

To be done as a group:

Write a two-column short paper for your project. You may reuse text from your topic survey, proposal and status report.

No need to submit project code.

Page Limit:

Four pages

Final Project Deliverables

Lightning Talk Presentation Video

Due: Friday 5/09

To be done as a group:

Record a video that presents a high-level introduction of your project. It can be abstract and is only meant to serve as an elevator pitch to incite interest and give an idea of what problem you are addressing and how you go about solving it.

e.g., <https://www.youtube.com/watch?v=oHWtQCGRkoo>

Time Limit:

90 seconds